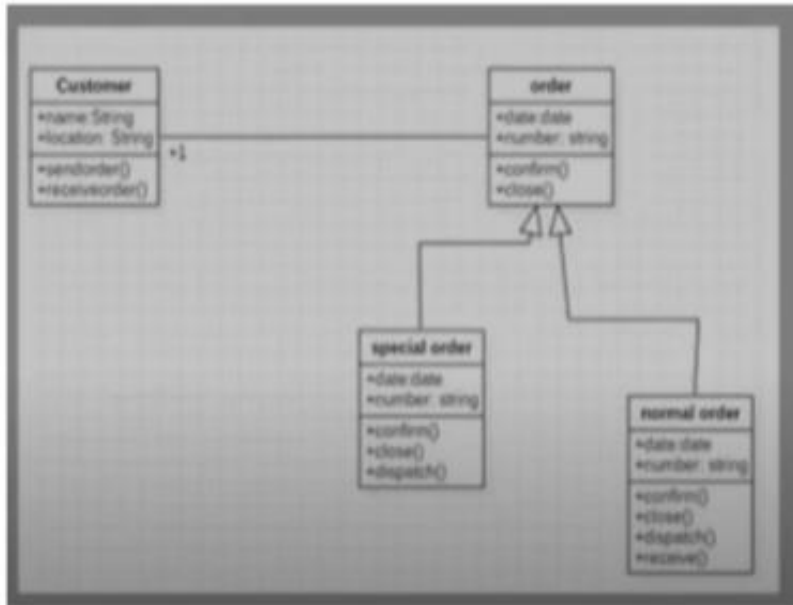


Customer demo problem

Aim: To write a Java program to demonstrate inheritance by creating a base Order class and deriving SpecialOrder and NormalOrder classes from it, while performing different order operations such as confirming, dispatching, receiving, and closing orders.



System Requirements :

Hardware:

- Standard PC/Laptop
- Intel Core i3/i5 Processor
- 4 GB or 8 GB RAM

Software:

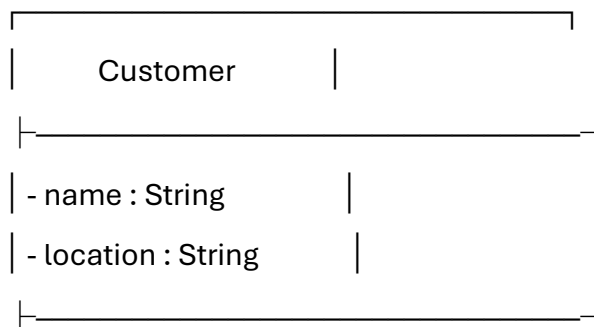
- Windows/Linux/macOS
- JDK 17 or JDK 21
- Eclipse / IntelliJ IDEA / NetBeans / VS Code

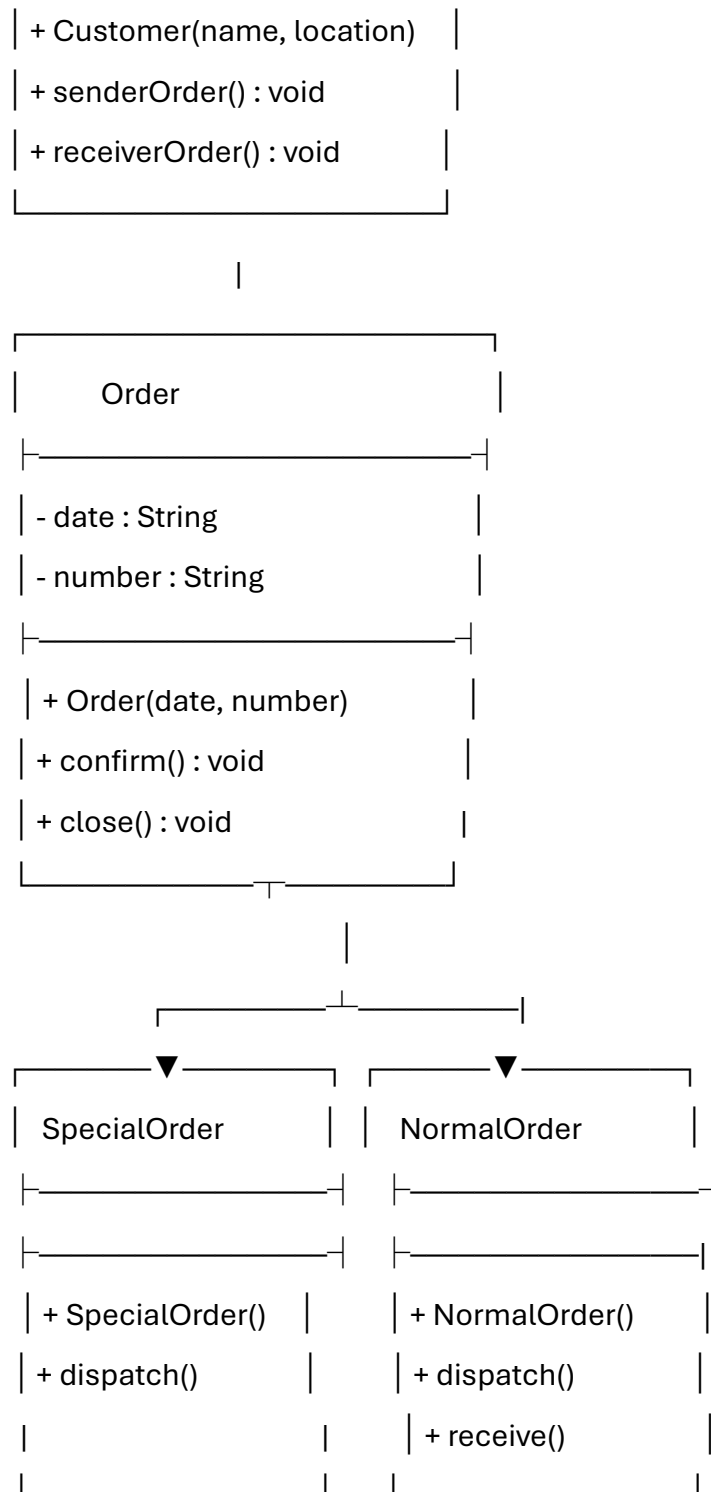
Algorithm:

1. Start
2. Define the Customer class with name and location.
3. Define the senderOrder() and receiverOrder() methods in the Customer class.
4. Define the Order class with date and number.

5. Define the `confirm()` and `close()` methods in the `Order` class.
6. Create the `SpecialOrder` class by inheriting the `Order` class.
7. Define the `dispatch()` method for `SpecialOrder`.
8. Create the `NormalOrder` class by inheriting the `Order` class.
9. Define the `dispatch()` and `receive()` methods for `NormalOrder`.
10. In the `main()` method, create a `Customer` object with name and location.
11. Create a `SpecialOrder` object with date and order number.
12. Create a `NormalOrder` object with date and order number.
13. Display the customer details.
 - For the special order:
 - Confirm the order.
 - Dispatch the order.
 - close the order.
 - For the normal order:
 - Confirm the order.
 - Dispatch the order.
 - Receive the order.
 - Close the order.
14. Call `senderOrder()` and `receiverOrder()` for the customer.
15. Stop.

UML Diagram:





Source code:

```

class Customer {
    String name;
    String location;

    // Constructor to initialize customer details

```

```
Customer(String name, String location) {  
    this.name = name;  
    this.location = location;  
}  
void senderOrder() {  
    System.out.println(name + " is sending an order.");  
}  
void receiverOrder() {  
    System.out.println(name + " is receiving an order.");  
}  
}  
// Order is the parent class  
class Order {  
    String date;  
    String number;  
    Order(String date, String number) {  
        this.date = date;  
        this.number = number;  
    }  
    void confirm() {  
        System.out.println("Order " + number + " is confirmed.");  
    }  
    void close() {  
        System.out.println("Order " + number + " is closed.");  
    }  
}  
// SpecialOrder inherits from Order  
class SpecialOrder extends Order {
```

```

SpecialOrder(String date, String number) {
    super(date, number);
}

void dispatch() {
    System.out.println("Special order " + number + " is dispatched.");
}
}

// NormalOrder inherits from Order
class NormalOrder extends Order {
    NormalOrder(String date, String number) {
        super(date, number);
    }

    void dispatch() {
        System.out.println("Normal order " + number + " is dispatched.");
    }

    void receive() {
        System.out.println("Normal order " + number + " is received.");
    }
}

public class OrderDemo {
    public static void main(String[] args) {
        Customer customer = new Customer("Meghana", "Srikakulam");
        SpecialOrder specialOrder = new SpecialOrder("11-08-2026", "S101");
        NormalOrder normalOrder = new NormalOrder("11-08-2026", "N101");

        // Displaying customer details
        System.out.println("Customer Details");
        System.out.println("Name   : " + customer.name);
        System.out.println("Location : " + customer.location);
    }
}

```

```
// Performing special order operations

    System.out.println("\nSpecial Order");

    specialOrder.confirm();

    specialOrder.dispatch();

    specialOrder.close();

// Performing normal order operations

    System.out.println("\nNormal Order");

    normalOrder.confirm();

    normalOrder.dispatch();

    normalOrder.receive();

    normalOrder.close();

// Performing customer order operations

    System.out.println();

    customer.senderOrder();

    customer.receiverOrder();

}

}
```

Compilation & Execution Commands :

Save the file as: OrderDemo.java

Compile : javac OrderDemo.java

Run: java OrderDemo

Output:

```
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac OrderDemo.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java OrderDemo
Customer Details
Name      : Meghana
Location  : Srikakulam

Special Order
Order S101 is confirmed.
Special order S101 is dispatched.
Order S101 is closed.

Normal Order
Order N101 is confirmed.
Normal order N101 is dispatched.
Normal order N101 is received.
Order N101 is closed.

Meghana is sending an order.
Meghana is receiving an order.
```

Result :

Thus, the Java program based on the given Order Management class diagram was successfully implemented and executed using classes, objects, inheritance and methods.